

Boosting I

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Gradient Boosting
- 5 Hyperparameters
- 6 Conclusions
- 7 Coming Up
- 8 Exercises and References

Recap

- Last time we covered **K -Nearest Neighbors**:
 - Procedure.
 - Selection of k .
 - Handling missing observations.

Introduction

- Random forests are an ensemble method involve aggregating predictions from many deep trees.
 - Very useful for reducing variance.
- Another method in ensemble learning involves sequencing *weak learners* to improve overall predictions.
 - At each iteration, each model learns and improves on the previous one.
- When done correctly, the final result is a very powerful ensemble.

Boosting Introduction

- **Boosting** is an ensemble method that improves accuracy by training weak learners sequentially, with each new model attempting to correct the errors of the previous one.
- Boosting is well suited for low-variance and high-bias scenarios.
 - Different than random forests!
- This approach may be applied to any single predictive model.
 - **We will focus on applying this to tree methods in this course.**

Boosting Procedure

- 1 Fit a (typically shallow) tree to the sample: $F_1(x) = y$
- 2 Fit the next (also shallow) tree to the the residuals of the previous tree: $h_1(x) = y - F_1(x)$
- 3 Add this tree (sequentially) to the algorithm: $F_2(x) = F_1(x) + h_1(x)$
- 4 Fit the next tree to the residuals of F_2 : $h_2(x) = y - F_2(x)$
- 5 Add the new tree: $F_3(x) = F_2(x) + h_2(x)$
- 6 Repeat this process until the stopping mechanism is reached.

$$f(x) = \sum_{b=1}^B f^b(x)$$

Illustrative Example 1

- Run the code to examine how the boosting process works.
- Note:
 - The blue line is the population function.
 - The red line is the current *boosted* model.
 - The black line is the current tree (fit to the residuals).
- What do you notice?

Gradient Decent

- Many of the algorithms we have covered focus on minimizing some function of the residuals.
 - In this scenario, the gradient is the residual error **but other loss functions may be useful.**
- The term **gradient boosting machine** comes from the fact that this approach may be applied to many loss functions.
- The general idea of gradient descent is to tweak parameter(s) iteratively in order to minimize a cost function.

Stochastic Gradient Decent

- An important parameter in gradient descent is the size of the steps which is controlled by the *learning rate*.
 - If the learning rate is too small, then the algorithm will take many iterations to find the minimum.
 - If the learning rate is too high, you might jump across the minimum and end up further away than when you started.
- Not all loss functions are nicely convex!
 - There may be local minima, plateaus, and other irregular terrain making finding a true minimum difficult.
- **Stochastic gradient descent** samples a fraction of the training observations (typically without replacement) and grows the next tree using that subsample.
 - Will not find a *true* minimum, but can help the algorithm jump out of local minima or off of plateaus.

GBM in R

- We can implement a basic GBM regression model from the gbm package.
 - In Python

- Usage:

```
model.gbm<- gbm(  
  formula = response ~ .,  
  data = data.train,  
  distribution = "gaussian", - SSE loss function  
  n.trees = 5000,  
  shrinkage = 0.1,  
  interaction.depth = 3,  
  n.minobsinnode = 10,  
  cv.folds = 10)
```

```
best <- which.min(model.gbm$cv.error) - Best number of trees  
sqrt(model.gbm$cv.error[best]) - compute RMSE  
gbm.perf(model.gbm, method = "cv")
```

Illustrative Example 2

- 1 Load the *iris* dataset into your environment and perform the train/test split.
- 2 Run the provided code to construct a gradient boosting machine for the Petal Length.
- 3 How many trees are deemed optimal?
- 4 Does our model predict the test set well?

Example 1

- Import the *Wages.csv* file to conduct the following analysis:
 - ① Perform a 75/25 train/test split.
 - ② Adjust the code from Illustrative Example 1 to construct a GBM model with *Salary* as the response variable (ignore the ID column).
 - ③ What is the *optimal* number of trees?
 - ④ Plot the progression of the number of trees.
 - ⑤ Does our model predict the test set well?
- What did you notice?

Hyperparameters

Hyperparameters

- A simple GBM model contains two categories of hyperparameters:
 - ① Boosting hyperparameters.
 - ② Tree-specific hyperparameters.
- Grid searches for these hyperparameters may be computationally demanding.

(1) Boosting Hyperparameters

- **Number of trees:**

- The total number of trees in the sequence or ensemble.
- It is easy to overfit these kinds of models.
- May still have many trees.
- Obtain the *optimal* number of trees through cross validation.

- **Learning rate:**

- Determines the contribution of each tree on the final outcome and controls how quickly the algorithm proceeds down the gradient descent.
- Values range from 0–1 with typical values between 0.001–0.3.
- Smaller learning rates (shrinkage) improve generalization and reduce overfitting, but require more trees and increase computation.
- Larger learning rates train faster but risk overfitting and poorer accuracy, while smaller values may not reach the optimum with a fixed number of trees.

(2) Tree-specific Hyperparameters

- **Tree depth:**

- Controls the depth of the individual trees (typically 3-8).
- Shallow trees are computationally efficient but require more trees.
- Deeper trees can capture more from the data but risk overfitting.
 - Deeper trees work better on large datasets.

- **Minimum number of observations in terminal nodes:**

- Also controls the complexity of each tree (early stopping).
- Typical values range from 5-15.
- Smaller values can help with class imbalances.
- *Rarely has a large impact on model performance.*

Tuning Strategy

- 1 Choose a relatively high learning rate. Generally the default value of 0.1 works but somewhere between 0.05–0.2 should work across a wide range of problems.
- 2 Determine the optimum number of trees for this learning rate.
- 3 Fix tree hyperparameters and tune learning rate and assess speed vs. performance.
- 4 Tune tree-specific parameters for decided learning rate.
- 5 Once tree-specific parameters have been found, lower the learning rate to assess for any improvements in accuracy.
- 6 Use final hyperparameter settings and increase CV procedures to get more robust estimates.

Illustrative Example 3 A

- Using the iris training data from Illustrative Example 2 do the following:
 - ① Run the code to perform a grid search for an optimal learning rate.
 - ② Save and print the results.
- What do you notice?

Illustrative Example 3 B

- Using the *optimal* learning rate from Illustrative Example 3 A do the following:
 - ① Run the code to perform a grid search for an optimal tree depth and minimum number of observations in the nodes.
 - ② Save and print the results.
 - ③ Does our model generalize better than Illustrative Example 2?
- What do you notice?

Example 2

- Using your data from Example 1 conduct the following analysis:
 - ① Adjust the the code from Illustrative Example 3 A to perform a grid search for an optimal learning rate.
 - ② Save and print the results.
 - ③ Adjust the code from Illustrative Example 3 B to perform a grid search for an optimal tree depth and minimum number of observations in the nodes.
 - ④ Save and print the results.
 - ⑤ Does our model generalize better than Example 1?
- What did you notice?

Final Thoughts

- GBMs can be very accurate prediction models.
- Hyperparameter tuning can be very computationally demanding.
 - Tune other parameters to specific learning rate.
- We may be able to make further improvements.

Next Lecture

- We will discuss **Boosting II**.
- Stochastic GBMs.
- XGBoost.
- Variable importance.

References & Resources

- ① Bakker, J. D. (2026) *Applied Multivariate Statistics in R*
 - ② Peng R. D. (2020) *Exploratory Data Analysis with R*
 - ③ Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
 - ④ Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R*. Taylor & Francis
- caret
 - gbm
 - gbm()